I've primarily selected these implementations to demonstrate particular techniques rather than to be state-of-the-art implementations; more advanced implementations that make better use of the available hardware concurrency may be found in the academic literature on parallel algorithms or in specialist multithreading libraries such as Intel's Threading Building Blocks.[4] The simplest parallel algorithm conceptually is a parallel version of `std::for_each`, so we'll start with that.

[4] http://threadingbuildingblocks.org/

8.5.1. A parallel implementation of std::for_each

`std::for_each` is simple in concept; it calls a user-supplied function on every element in a range in turn. The big difference between a parallel implementation and the sequential `std::for_each` is the order of the function calls. `std::for_each` calls the function with the first element in the range, then the second, and so on, whereas with a parallel implementation there's no guarantee as to the order in which the elements will be processed, and they may (indeed we hope they *will*) be processed concurrently.

To implement a parallel version of this, you just need to divide the range into sets of elements to process on each thread. You know the number of elements in advance, so you can divide the data before processing begins (section 8.1.1). We'll assume that this is the only parallel task running, so you can use `std::thread::hardware_concurrency()` to determine the number of threads. You also know that the elements can be processed entirely independently, so you can use contiguous blocks to avoid false sharing (section 8.2.3).

This algorithm is similar in concept to the parallel version of `std::accumulate` described in section 8.4.1, but rather than computing the sum of each element, you merely have to apply the specified function. Although you might imagine this would greatly simplify the code, because there's no result to return, if you wish to pass on exceptions to the caller, you still need to use the `std::packaged_task` and `std::future`